

Spring Data JPA & Hibernate — Hands-on Answers

Doc 1 — Spring Data JPA Example (orm-learn / Country)

Hands on 1: Spring Data JPA — Example

application.properties

```
logging.level.org.springframework=info
logging.level.com.cognizant=debug
logging.level.org.hibernate.SQL=trace
logging.level.org.hibernate.type.descriptor.sql=trace
logging.pattern.console=%d{dd-MM-yy} %d{HH:mm:ss.SSS} %-20.20thread %5p %-
25.25logger{25} %25M %4L %m%n

spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
spring.datasource.url=jdbc:mysql://localhost:3306/ormlearn
spring.datasource.username=root
spring.datasource.password=root

spring.jpa.hibernate.ddl-auto=validate
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL5Dialect
```

Country.java

```
package com.cognizant.ormlearn.model;

import javax.persistence.Column;
import javax.persistence.Entity;
import javax.persistence.Id;
import javax.persistence.Table;

@Entity
@Table(name = "country")
public class Country {

    @Id
    @Column(name = "co_code")
    private String code;

    @Column(name = "co_name")
    private String name;

    public String getCode() { return code; }
    public void setCode(String code) { this.code = code; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }

    @Override
    public String toString() {
        return "Country [code=" + code + ", name=" + name + "]";
    }
}
```

CountryRepository.java

```
package com.cognizant.ormlearn.repository;
```

```

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
import com.cognizant.ormlearn.model.Country;

@Repository
public interface CountryRepository extends JpaRepository<Country, String> {
}

```

CountryService.java

```

package com.cognizant.ormlearn.service;

import java.util.List;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import com.cognizant.ormlearn.model.Country;
import com.cognizant.ormlearn.repository.CountryRepository;

@Service
public class CountryService {

    @Autowired
    private CountryRepository countryRepository;

    @Transactional
    public List<Country> getAllCountries() {
        return countryRepository.findAll();
    }
}

```

OrmLearnApplication.java (test)

```

package com.cognizant.ormlearn;

import java.util.List;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.ApplicationContext;
import com.cognizant.ormlearn.model.Country;
import com.cognizant.ormlearn.service.CountryService;

@SpringBootApplication
public class OrmLearnApplication {

    private static final Logger LOGGER =
LoggerFactory.getLogger(OrmLearnApplication.class);
    private static CountryService countryService;

    public static void main(String[] args) {
        ApplicationContext context =
SpringApplication.run(OrmLearnApplication.class, args);
        countryService = context.getBean(CountryService.class);
        LOGGER.info("Inside main");
        testGetAllCountries();
    }
}

```

```
private static void testGetAllCountries() {
    LOGGER.info("Start");
    List<Country> countries = countryService.getAllCountries();
    LOGGER.debug("countries={}", countries);
    LOGGER.info("End");
}
}
```

Hands on 2: Hibernate XML Config Walkthrough

- Persistence class mapped through a separate <mapping> XML file (e.g. Employee.hbm.xml), which links class fields to table columns, instead of annotations.
- SessionFactory — heavy, one per application, built once from hibernate.cfg.xml; creates Session objects.
- Session — lightweight, one per unit-of-work / request, wraps a JDBC connection, used to save/get/delete.
- Transaction — obtained via session.beginTransaction(); tx.commit() persists changes, tx.rollback() undoes them on failure.
- session.save(obj) inserts, session.get(Class, id) fetches by primary key, session.createQuery(hql).list() runs HQL, session.delete(obj) removes a row.

Hands on 3: Hibernate Annotation Config Walkthrough

- @Entity marks the class as a persistence class; @Table(name="employee") maps it to the table.
- @Id marks the primary key field; @GeneratedValue(strategy = GenerationType.IDENTITY) lets the DB auto-increment it.
- @Column(name="...") maps a field to a specific column name.
- hibernate.cfg.xml still holds the connection details even in annotation config: dialect, driver_class, connection.url, connection.username, connection.password, plus <mapping class="..."> instead of an XML mapping file.

Hands on 4: JPA vs Hibernate vs Spring Data JPA

- JPA — just a specification for how Java objects map to relational data; it has no implementation of its own.
- Hibernate — an ORM tool that implements the JPA specification (one of several possible implementations).
- Spring Data JPA — sits above Hibernate/JPA, removes boilerplate DAO code, manages transactions, and gives repository interfaces like JpaRepository out of the box.

Code comparison — Hibernate vs Spring Data JPA

```
// Hibernate style
public Integer addEmployee(Employee employee) {
    Session session = factory.openSession();
    Transaction tx = null;
    Integer employeeID = null;
    try {
        tx = session.beginTransaction();
        employeeID = (Integer) session.save(employee);
        tx.commit();
    }
```

```

    } catch (HibernateException e) {
        if(tx!=null)tx.rollback();
        e.printStackTrace();
    } finally {
        session.close();
    }
    return employeeID;
}

// Spring Data JPA style
public interface EmployeeRepository extends JpaRepository<Employee, Integer> {
}

@Autowired
private EmployeeRepository employeeRepository;

@Transactional
public void addEmployee(Employee employee) {
    employeeRepository.save(employee);
}

```

Hands on 5: ddl-auto & Populating Country Table

- create — drops and recreates tables every run (data lost).
- validate — only checks schema matches entities, throws an error if not; does not alter anything.
- update — adds missing tables/columns without dropping existing data.
- create-drop — creates schema at startup, drops it when the app shuts down.

Ran the given insert script against ormlearn.country after clearing existing rows - this became the base data for Hands on 6–9.

Hands on 6: Find Country by Code

CountryNotFoundException.java

```

package com.cognizant.ormlearn.service.exception;

public class CountryNotFoundException extends Exception {
    public CountryNotFoundException(String message) {
        super(message);
    }
}

```

CountryService — findCountryByCode()

```

@Transactional
public Country findCountryByCode(String countryCode) throws
CountryNotFoundException {
    Optional<Country> result = countryRepository.findById(countryCode);
    if (!result.isPresent()) {
        throw new CountryNotFoundException("Country not found for code: " +
countryCode);
    }
    return result.get();
}

```

Test in OrmLearnApplication

```
private static void getAllCountriesTest() throws CountryNotFoundException {
    LOGGER.info("Start");
    Country country = countryService.findCountryByCode("IN");
    LOGGER.debug("Country:{}", country);
    LOGGER.info("End");
}
```

Hands on 7: Add a New Country

```
@Transactional
public void addCountry(Country country) {
    countryRepository.save(country);
}

private static void testAddCountry() throws CountryNotFoundException {
    Country country = new Country();
    country.setCode("XX");
    country.setName("Test Country");
    countryService.addCountry(country);

    Country saved = countryService.findCountryByCode("XX");
    LOGGER.debug("Saved country:{}", saved);
}
```

Hands on 8: Update a Country

```
@Transactional
public void updateCountry(String code, String name) {
    Country country = countryRepository.findById(code).get();
    country.setName(name);
    countryRepository.save(country);
}

private static void testUpdateCountry() {
    countryService.updateCountry("XX", "Renamed Country");
}
```

Hands on 9: Delete a Country

```
@Transactional
public void deleteCountry(String code) {
    countryRepository.deleteById(code);
}

private static void testDeleteCountry() {
    countryService.deleteCountry("XX");
}
```

Doc 2 — Employee Management System

Exercise 1: Project Setup

- Spring Initializr: Group com.cognizant, Artifact EmployeeManagementSystem, Dependencies: Spring Web, Spring Data JPA, H2 Database, Lombok.

application.properties

```
spring.datasource.url=jdbc:h2:mem:testdb
spring.datasource.driverClassName=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=password
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
spring.jpa.hibernate.ddl-auto=update
spring.h2.console.enabled=true
```

Exercise 2: Entities

Department.java

```
package com.cognizant.ems.model;

import java.util.List;
import javax.persistence.*;
import lombok.Data;

@Entity
@Table(name = "department")
@Data
public class Department {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    @OneToMany(mappedBy = "department", cascade = CascadeType.ALL)
    private List<Employee> employees;
}
```

Employee.java

```
package com.cognizant.ems.model;

import javax.persistence.*;
import lombok.Data;

@Entity
@Table(name = "employee")
@Data
public class Employee {

    @Id
```

```

    @GeneratedValue(strategy=GenerationType.IDENTITY)
    private Long id;

    private String name;
    private String email;

    @ManyToOne
    @JoinColumn(name="department_id")
    private Department department;
}

```

Exercise 3: Repositories

```

public interface EmployeeRepository extends JpaRepository<Employee, Long> {
    List<Employee> findByDepartmentId(Long departmentId);
    List<Employee> findByNameContainingIgnoreCase(String name);
}

public interface DepartmentRepository extends JpaRepository<Department, Long> {
    Optional<Department> findByName(String name);
}

```

Exercise 4: CRUD REST Endpoints

EmployeeController.java

```

@RestController
@RequestMapping("/employees")
public class EmployeeController {

    @Autowired
    private EmployeeRepository employeeRepository;

    @GetMapping
    public List<Employee> getAll() {
        return employeeRepository.findAll();
    }

    @GetMapping("/{id}")
    public Employee getById(@PathVariable Long id) {
        return employeeRepository.findById(id).orElse(null);
    }

    @PostMapping
    public Employee create(@RequestBody Employee employee) {
        return employeeRepository.save(employee);
    }

    @PutMapping("/{id}")
    public Employee update(@PathVariable Long id, @RequestBody Employee employee)
    {
        employee.setId(id);
        return employeeRepository.save(employee);
    }

    @DeleteMapping("/{id}")
}

```



```

        public void delete(@PathVariable Long id) {
            employeeRepository.deleteById(id);
        }
    }
}

```

DepartmentController.java

```

@RestController
@RequestMapping("/departments")
public class DepartmentController {

    @Autowired
    private DepartmentRepository departmentRepository;

    @GetMapping
    public List<Department> getAll() { return departmentRepository.findAll(); }

    @PostMapping
    public Department create(@RequestBody Department department) {
        return departmentRepository.save(department);
    }

    @PutMapping("/{id}")
    public Department update(@PathVariable Long id, @RequestBody Department
department) {
        department.setId(id);
        return departmentRepository.save(department);
    }

    @DeleteMapping("/{id}")
    public void delete(@PathVariable Long id)
{ departmentRepository.deleteById(id); }
}

```

Exercise 5: Query Methods & Named Queries

```

public interface EmployeeRepository extends JpaRepository<Employee, Long> {

    List<Employee> findByEmailEndingWith(String domain);

    @Query("SELECT e FROM Employee e WHERE e.department.name = :deptName")
    List<Employee> findByDepartmentName(@Param("deptName") String deptName);
}

@Entity
@NamedQueries({
    @NamedQuery(name = "Employee.findByName",
        query = "SELECT e FROM Employee e WHERE e.name = :name")
})
public class Employee { /* ... */ }

```

Exercise 6: Pagination & Sorting

```

public interface EmployeeRepository extends JpaRepository<Employee, Long> {
    Page<Employee> findAll(Pageable pageable);
}

```

```

@GetMapping("/search")
public Page<Employee> search(
    @RequestParam(defaultValue = "0") int page,
    @RequestParam(defaultValue="10") int size,
    @RequestParam(defaultValue = "name") String sortBy) {
    Pageable pageable = PageRequest.of(page, size, Sort.by(sortBy).ascending());
    return employeeRepository.findAll(pageable);
}

```

Exercise 7: Entity Auditing

```

@SpringBootApplication
@EnableJpaAuditing
public class EmployeeManagementSystemApplication { /* ... */ }

@Entity
@EntityListeners(AuditingEntityListener.class)
public class Employee {
    // existing fields ...

    @CreatedDate
    private LocalDateTime createdAt;

    @LastModifiedDate
    private LocalDateTime lastModifiedDate;

    @CreatedBy
    private String createdBy;

    @LastModifiedBy
    private String lastModifiedBy;
}

```

Exercise 8: Projections

Interface-based projection

```

public interface EmployeeNameOnly {
    String getName();
    String getEmail();
}

public interface EmployeeRepository extends JpaRepository<Employee, Long> {
    List<EmployeeNameOnly> findByDepartmentId(Long departmentId);
}

```

Class-based (DTO) projection

```

public class EmployeeSummary {
    private final String name;
    private final String departmentName;

    public EmployeeSummary(String name, String departmentName) {
        this.name = name;
        this.departmentName = departmentName;
    }
    // getters
}

```

```

}

@Query("SELECT new com.cognizant.ems.dto.EmployeeSummary(e.name,
e.department.name) FROM Employee e")
List<EmployeeSummary> findEmployeeSummaries();

```

Exercise 9: Data Source Configuration

```

# application.properties - primary + secondary datasource
spring.datasource.primary.url=jdbc:h2:mem:primarydb
spring.datasource.primary.username=sa
spring.datasource.primary.password=password

spring.datasource.secondary.url=jdbc:h2:mem:secondarydb
spring.datasource.secondary.username=sa
spring.datasource.secondary.password=password

@Configuration
public class DataSourceConfig {

    @Primary
    @Bean
    @ConfigurationProperties(prefix = "spring.datasource.primary")
    public DataSource primaryDataSource() {
        return DataSourceBuilder.create().build();
    }

    @Bean
    @ConfigurationProperties(prefix = "spring.datasource.secondary")
    public DataSource secondaryDataSource() {
        return DataSourceBuilder.create().build();
    }
}

```

Exercise 10: Hibernate-Specific Features

```

@Entity
@DynamicUpdate // only updates changed columns
@DynamicInsert // skips null columns on insert
public class Employee { /* ... */ }

# Hibernate dialect + batch processing
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.H2Dialect
spring.jpa.properties.hibernate.jdbc.batch_size=30
spring.jpa.properties.hibernate.order_inserts=true
spring.jpa.properties.hibernate.order_updates=true

```

Doc 3 — HQL, JPQL & Criteria Query

Hands on 1: HQL vs JPQL

- HQL (Hibernate Query Language) and JPQL (Java Persistence Query Language) both query against entity objects, not raw tables.
- JPQL is a subset of HQL — every valid JPQL query is valid HQL, not the other way round.
- Both support SELECT, UPDATE, DELETE; HQL additionally supports INSERT.

Hands on 2: Get All Permanent Employees (HQL)

Naive version (multiple queries / N+1 issue)

```
@Query(value = "SELECT e FROM Employee e WHERE e.permanent = 1")
List<Employee> getAllPermanentEmployees();
```

Optimized version — using fetch to avoid extra queries

```
@Query(value = "SELECT e FROM Employee e left join fetch e.department d "
    + "left join fetch e.skillList WHERE e.permanent = 1")
List<Employee> getAllPermanentEmployees();
```

Service + test

```
@Transactional
public List<Employee> getAllPermanentEmployees() {
    return employeeRepository.getAllPermanentEmployees();
}

public static void testGetAllPermanentEmployees() {
    LOGGER.info("Start");
    List<Employee> employees = employeeService.getAllPermanentEmployees();
    LOGGER.debug("Permanent Employees:{}", employees);
    employees.forEach(e -> LOGGER.debug("Skills:{}", e.getSkillList()));
    LOGGER.info("End");
}
```

Key takeaway: join links tables but doesn't populate the collection; fetch is what actually loads the data, so use it only where the data is really needed.

Hands on 3: Fetch Quiz Attempt Details (HQL)

AttemptRepository.java

```
public interface AttemptRepository extends JpaRepository<Attempt, Integer> {

    @Query("SELECT a FROM Attempt a "
        + "join fetch a.user u "
        + "join fetch a.attemptQuestionList aq "
        + "join fetch aq.question q "
        + "join fetch aq.attemptOptionList ao "
        + "join fetch ao.option o "
        + "WHERE u.id = :userId AND a.id = :attemptId")
```

```

    Attempt getAttempt(@Param("userId") int userId, @Param("attemptId") int attemptId);
}

```

AttemptService.java

```

@Transactional
public Attempt getAttempt(int userId, int attemptId) {
    return attemptRepository.getAttempt(userId, attemptId);
}

```

Test in OrmLearnApplication

```

private static void testGetAttemptDetail() {
    Attempt attempt = attemptService.getAttempt(1, 1);
    attempt.getAttemptQuestionList().forEach(aq -> {
        LOGGER.debug(aq.getQuestion().getText());
        aq.getAttemptOptionList().forEach(ao ->
            LOGGER.debug("{} {} {} {}",
                ao.getOption().getId(), ao.getOption().getText(),
                ao.getOption().getScore(), ao.isSelected()));
    });
}

```

Hands on 4: Average Salary (HQL)

```

@Query(value = "SELECT AVG(e.salary) FROM Employee e")
double getAverageSalary();

```

```

@Query(value = "SELECT AVG(e.salary) FROM Employee e where e.department.id = :id")
double getAverageSalary(@Param("id") int id);

```

Note: `e.department.id` navigates through the ManyToOne relationship on `Employee`; `:id` is bound via `@Param`; any aggregate function (SUM, MAX, MIN, COUNT) works the same way.

Hands on 5: Get All Employees (Native Query)

```

@Query(value = "SELECT * FROM employee", nativeQuery = true)
List<Employee> getAllEmployeesNative();

```

Kept this to a minimum since native queries tie the code to a specific database and lose the portability HQL/JPQL gives.

Hands on 6: Criteria Query

For the Amazon-style filter scenario, the WHERE clause changes depending on which filters (review, HDD, RAM, CPU speed, OS, weight) the user actually picked — so it can't be a fixed HQL string. `CriteriaBuilder` lets predicates be added conditionally at runtime.

```

CriteriaBuilder cb = entityManager.getCriteriaBuilder();
CriteriaQuery<Product> cq = cb.createQuery(Product.class);
Root<Product> product = cq.from(Product.class);

```

```

List<Predicate> predicates = new ArrayList<>();
predicates.add(cb.like(product.get("name"), "%laptop%"));

```

```
if (minRam != null) {
    predicates.add(cb.greaterThanOrEqualTo(product.get("ram"), minRam));
}
if (operatingSystem != null) {
    predicates.add(cb.equal(product.get("os"), operatingSystem));
}
if (maxWeight != null) {
    predicates.add(cb.lessThanOrEqualTo(product.get("weight"), maxWeight));
}

cq.select(product).where(predicates.toArray(new Predicate[0]));
TypedQuery<Product> query = entityManager.createQuery(cq);
List<Product> results = query.getResultList();
```

Doc 4 — Query Methods & O/R Mapping

Hands on 1: Query Methods on Country Table

Countries containing a substring

```
public interface CountryRepository extends JpaRepository<Country, String> {  
    List<Country> findByNameContaining(String text);  
}
```

Same query, ascending order by name

```
List<Country> findByNameContainingOrderByNameAsc(String text);
```

Countries starting with a letter

```
List<Country> findByNameStartingWith(String letter);
```

Test in OrmLearnApplication

```
private static void testCountryQueries() {  
    countryRepository.findByNameContaining("ou").forEach(c -> LOGGER.debug("{} ",  
c));  
    countryRepository.findByNameContainingOrderByNameAsc("ou").forEach(c ->  
LOGGER.debug("{} ", c));  
    countryRepository.findByNameStartingWith("Z").forEach(c -> LOGGER.debug("{} ",  
c));  
}
```

Hands on 2: Query Methods on Stock Table

Stock.java

```
@Entity  
@Table(name = "stock")  
public class Stock {  
  
    @Id  
    @GeneratedValue(strategy = GenerationType.IDENTITY)  
    @Column(name = "st_id")  
    private int id;  
  
    @Column(name = "st_code")  
    private String code;  
  
    @Column(name = "st_date")  
    private LocalDate date;  
  
    @Column(name = "st_open")  
    private double open;  
  
    @Column(name = "st_close")  
    private double close;  
  
    @Column(name = "st_volume")  
    private long volume;
```

```

    // getters, setters, toString()
}

```

StockRepository.java — required query methods

```

public interface StockRepository extends JpaRepository<Stock, Integer> {

    // Facebook stock details for September 2019
    List<Stock> findByCodeAndDateBetween(String code, LocalDate startDate,
LocalDate endDate);

    // Google stock where open price greater than 1250
    List<Stock> findByCodeAndOpenGreaterThan(String code, double open);

    // Top 3 dates with highest volume
    List<Stock> findTop3ByCodeOrderByVolumeDesc(String code);

    // Lowest 3 close prices for Netflix
    List<Stock> findTop3ByCodeOrderByCloseAsc(String code);
}

```

Test in OrmLearnApplication

```

private static void testStockQueries() {
    stockRepository.findByCodeAndDateBetween("FB",
        LocalDate.of(2019, 9, 1), LocalDate.of(2019, 9, 30))
        .forEach(s -> LOGGER.debug("{} ", s));

    stockRepository.findByCodeAndOpenGreaterThan("GOOGL", 1250)
        .forEach(s -> LOGGER.debug("{} ", s));

    stockRepository.findTop3ByCodeOrderByVolumeDesc("FB")
        .forEach(s -> LOGGER.debug("{} ", s));

    stockRepository.findTop3ByCodeOrderByCloseAsc("NFLX")
        .forEach(s -> LOGGER.debug("{} ", s));
}

```

Hands on 3: Payroll Schema — Entities & Repositories

```

@Entity
@Table(name = "employee")
public class Employee {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "em_id")
    private int id;

    @Column(name = "em_name")
    private String name;

    @Column(name = "em_salary")
    private double salary;

    @Column(name = "em_permanent")
    private boolean permanent;
}

```



```

        @Column(name="em_date_of_birth")
        private Date dateOfBirth;
        // getters, setters, toString()
    }

@Entity
@Table(name="department") public
class Department {
    @Id
    @GeneratedValue(strategy=GenerationType.IDENTITY)
    @Column(name = "dp_id")
    private int id;

    @Column(name="dp_name")
    private String name;
    // getters, setters, toString()
}

@Entity
@Table(name="skill") public
class Skill {
    @Id
    @GeneratedValue(strategy=GenerationType.IDENTITY)
    @Column(name = "sk_id")
    private int id;

    @Column(name="sk_name")
    private String name;
    // getters, setters, toString()
}

public interface EmployeeRepository extends JpaRepository<Employee, Integer> {} public
interface DepartmentRepository extends JpaRepository<Department, Integer>
{ }
public interface SkillRepository extends JpaRepository<Skill, Integer> { }

```

Hands on 4: @ManyToOne — Employee to Department

```

@ManyToOne
@JoinColumn(name = "em_dp_id")
private Department department;
// getter/setter

```

EmployeeService.java

```

@Service
public class EmployeeService {

    @Autowired
    private EmployeeRepository employeeRepository;

    @Transactional
    public Employee get(int id) {
        return employeeRepository.findById(id).get();
    }

    @Transactional
    public void save(Employee employee) {

```

```

        employeeRepository.save(employee);
    }
}
// DepartmentService and SkillService follow the same get()/save() pattern

```

Test methods in OrmLearnApplication

```

private static void testGetEmployee() {
    Employee employee = employeeService.get(1);
    LOGGER.debug("Employee:{}", employee);
    LOGGER.debug("Department:{}", employee.getDepartment());
}

private static void testAddEmployee() {
    Employee employee = new Employee();
    employee.setName("New Employee");
    employee.setSalary(50000);
    employee.setPermanent(true);
    employee.setDepartment(departmentService.get(1));
    employeeService.save(employee);
    LOGGER.debug("Saved:{}", employee);
}

private static void testUpdateEmployee() {
    Employee employee = employeeService.get(1);
    employee.setDepartment(departmentService.get(2));
    employeeService.save(employee);
    LOGGER.debug("Updated:{}", employee);
}

```

Hands on 5: @OneToMany — Department to Employee

```

@OneToMany(mappedBy = "department", fetch = FetchType.EAGER)
private Set<Employee> employeeList;
// getter/setter

private static void testGetDepartment() {
    Department department = departmentService.get(2);
    LOGGER.debug("Department:{}", department);
    LOGGER.debug("Employees:{}", department.getEmployeeList());
}

```

With the default LAZY fetch, this throws LazyInitializationException outside the session — switching to fetch = FetchType.EAGER on employeeList fixes it.

Hands on 6: @ManyToMany — Employee and Skill

Employee.java

```

@ManyToMany(fetch = FetchType.EAGER)
@JoinTable(name = "employee_skill",
    joinColumns = @JoinColumn(name = "es_em_id"),
    inverseJoinColumns = @JoinColumn(name = "es_sk_id"))
private Set<Skill> skillList;
// getter/setter

```

Skill.java

```
@ManyToMany(mappedBy = "skillList")
private Set<Employee> employeeList;
// getter/setter
```

Add skill to employee — test

```
private static void testAddSkillToEmployee() {
    Employee employee = employeeService.get(1);
    Skill skill = skillService.get(3);
    employee.getSkillList().add(skill);
    employeeService.save(employee);
    LOGGER.debug("Employee skills now:{}", employee.getSkillList());
}
```